

2D LIDAR MAPPING

LiStars

Presented by

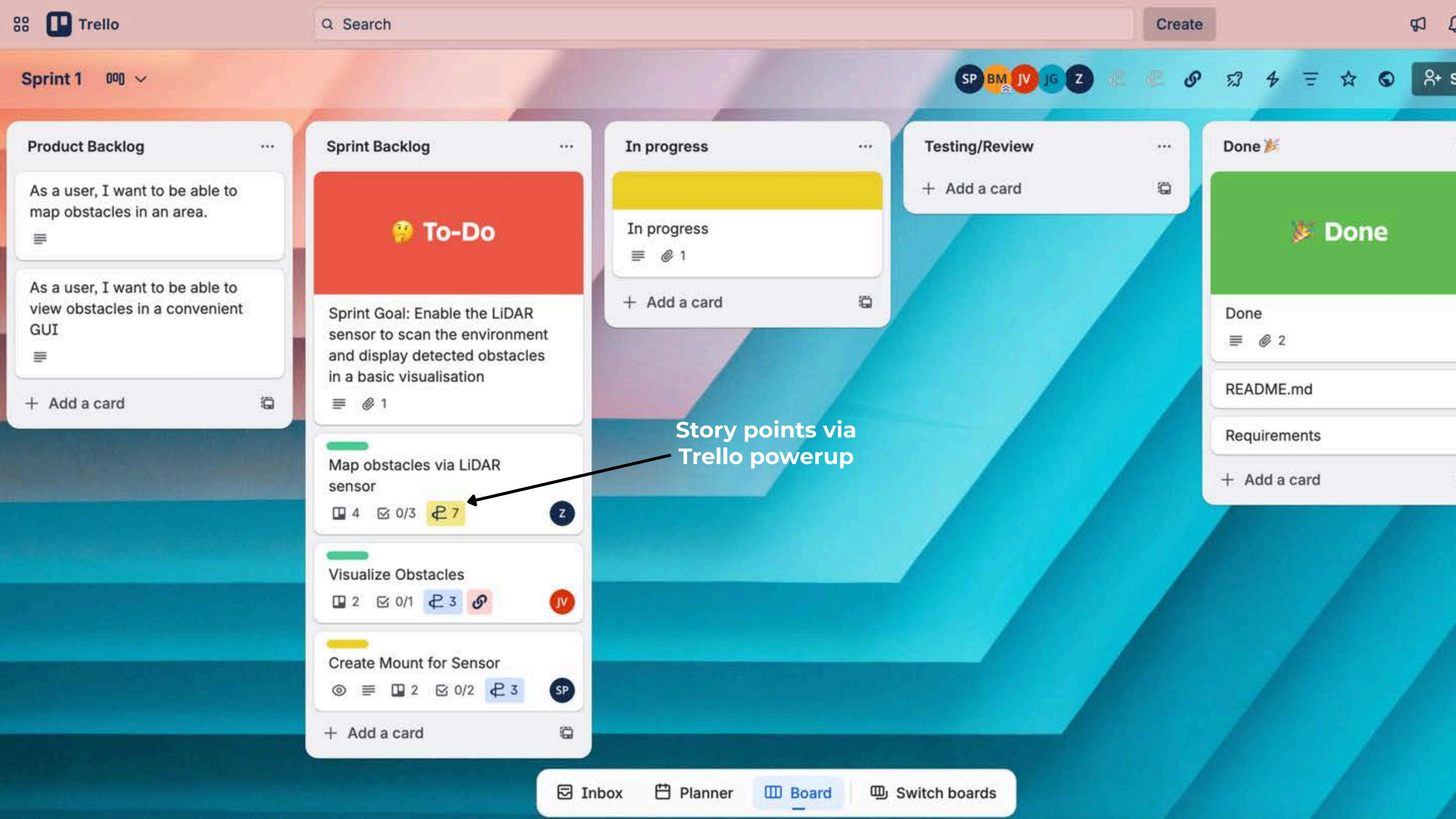
Zac Swisher, Sneha Patel, Jamil Velez, Bryton Montgomery

SPRINT 1: GOAL

Enable the LiDAR sensor to scan the environment and display detected obstacles in a basic visualization.

Selected Tasks:

- Must Have:
 - Map obstacles via LiDAR sensor
 - Visualize Obstacles
- Should Have:
 - Create Mount for Sensor
- Could have:
 - Assemble LiDAR Sensor & motor on 3D-printed mount



Story points via
Trello powerup

Task

**Assigned
Team
Member**

○ **Map obstacles via LiDAR sensor**

+ Add ⌚ Dates ☑ Checklist 📎 Attachment

Members Labels

z + Must Have +

☰ Description

Add a more detailed description...

🔗 Card Dependencies [Remove...](#)

Blocks	Data Export as CSV	by Jamil Velez
Blocks	Maps obstacles within radius 360 degrees	by unassigned
Blocks	Maps obstacles within radius 360 degrees	by zs
Blocks	Visualize Obstacles	by Jamil Velez

**Dependencies via Trello
Powerup**

☑ Checklist [Delete](#)

0%

☐ Get lidar sensor working

☐ create data pipeline script to retrieve data from sensor

☐ save raw data

[Add an item](#)

Subtasks

Task 1: Map Obstacles Via LiDAR Sensor

Why? This is the core functionality of the entire project. Without collecting obstacle data, nothing else (visualization, GUI, etc.) can progress. It establishes the data pipeline from hardware to software.

Subtasks:

- Get lidar sensor working
- Create data pipeline script to retrieve data from sensor
- Save raw data

Task 1: Map Obstacles Via LiDAR Sensor

Effort Estimation: 7 Story Points

Dependencies:

- **This task blocks:**

- Visualize Obstacles → Needs raw mapping data to display.
- Maps obstacles within radius 360 degrees → Requires basic mapping working first.
- Data Export as CSV → Needs collected data to export.

Task 2: Visualize Obstacles

Why? Once the LiDAR data is collected, we need a way to see what the sensor is detecting. This task turns raw data into something meaningful and usable.

Subtasks:

- Create Python script that takes in the raw obstacle data and displays obstacles in a GUI

Task 2: Visualize Obstacles

Effort Estimation: 2 Story Points

Dependencies:

- **This task is blocked by:**
 - Map obstacles via LiDAR sensor → Because visualization requires raw obstacle data to exist first.
- **This task blocks:**
 - Data Export as SVG → Because exporting a graphical representation depends on having a working visualization.

Task 3: Create Mount for Sensor

Why? The LiDAR sensor needs a stable mount to collect accurate data. This is a small but necessary hardware task that supports the mapping feature.

Subtasks:

- Design a mount for Sensor
- Print the mount via maker space

Task 3: Create Mount for Sensor

Effort Estimation: 2 story points

Dependencies:

- **This task blocks:**

- Assemble LiDAR Sensor & motor on 3D-printed mount

Task 4: Assemble LiDAR Sensor & motor on 3D-printed mount

Why? This task ensures the LiDAR sensor and motor are physically stable, accurately aligned, and capable of rotating.

Task 4: Assemble LiDAR Sensor & motor on 3D-printed mount

Effort Estimation: 1 story points

Dependencies:

- **This task is blocked by:**
 - Create Mount for Sensor → because the mount must exist before assembling the hardware.

SPRINT 2

Sprint Goal: Create a functional prototype that can scan obstacles while moving, is housed in a basic enclosure, and can export the collected data.

Selected Tasks:

- Must Have:
 - Assemble LiDAR Sensor & motor on 3D-printed mount
- Should Have:
 - Maps obstacles within radius 360 degrees

Outcome: By the end of Sprint 2, the system should physically rotate and generate accurate 360° obstacle maps.

SPRINT 3

Sprint Goal: Create a functional prototype that can scan obstacles while moving, is housed in a basic enclosure, and can export the collected data.

Selected Tasks:

- Must Have:
 - Maps obstacles within radius 360 degrees
 - Move mounted sensor in a basic path
- Should Have:
 - Data Export as SVG

Outcome: By the end of Sprint 3, you will have a portable, functioning prototype that scans, visualizes, and exports obstacle data in real use conditions.